COURSEPACK

SCHEME

The scheme is an overview of work-integrated learning opportunities and gets students out into the real world. This will give what a course entails.

Course Title	Design & Analysis of Algorithm			Course Type				Integrated	
Course Code	R1PC302B			Class				MCA III Sem	
Instruction delivery	Activity	Credits	Credit Hours	Total Number of Classes per Semester				Assessment in Weightage	
	Lecture	3	3						
	Tutorial	0	0	Theory	Tutorial	Practical	Self-study	CIE	SEE
	Practical	1	2						
	Self-study	0	0						
	Total	4	5	45	0	15	0	50%	50%
Course Lead	Dr. Sonia Rani		Course Coordinator	Dr. Anil Gankotia					
Names Course Instructors	Theory			Practical					
	Dr. Sanjeev Kumar Prasad Dr. Prashant Johri Dr.Unni Krishnan Dr Anil Gankotia Dr. Sonia Rani Dr. Sanjiv Sharma Dr. Bhoopendra Dewedi Dr. Anuj Kumar Singh Dr. Farhan Sufiyan			Dr. Sanjeev Kumar Prasad Dr. Prashant Johri Dr.Unni Krishnan Dr Anil Gankotia Dr. Sonia Rani Dr. Sanjiv Sharma Dr. Bhoopendra Dewedi Dr. Anuj Kumar Singh Dr. Farhan Sufiyan					

COURSE OVERVIEW

To introduce students, the concepts of algorithm analysis for finding out the space and time complexity of different algorithms. Different design techniques such as greedy method, divide and conquer, backtracking, dynamic programming, branch and bound are to be studied for finding the solution to the different problems. It also provides an insight into the basic concepts of NP and NP-hard problems and their relevance in research.

PREREQUISITE COURSE

Prerequisite Course Required	Yes

If, yes please fill in the Details	**Prerequisite Course code**	**Prerequisite course name**
		Data structure using C

COURSE OBJECTIVE

1. To develop proficiency in designing efficient algorithms for solving diverse computational problems.
2. To analyze the time and space complexity of algorithms to evaluate their performance.
3. To apply algorithmic strategies and techniques to improve problem-solving capabilities.
4. To gain insights into various algorithmic paradigms and their applications in real-world scenarios.
5. To demonstrate the ability to critically assess and compare different algorithms for specific tasks.

COURSE OUTCOMES (COs)

After the completion of the course, the student will be able to:

CO No.	Course Outcomes
R1PC302B.1	Understand the concept of algorithms.
R1PC302B.2	Analyze algorithmic complexity for efficient problem-solving.
R1PC302B.3	Apply algorithms to real-world scenarios.
R1PC302B.4	Develop problem-solving skills through algorithmic approaches.

BLOOM'S LEVEL OF THE COURSE OUTCOMES

Bloom's taxonomy is a set of hierarchical models used for the classification of educational learning objectives into levels of complexity and specificity. The learning domains are cognitive, affective, and psychomotor.

INTEGRATED

CO No.	Remember **KL1**	Understand **KL 2**	Apply **KL 3**	Analyse **KL 4**	Evaluate **KL 5**	Create **KL 6**

R1PC302B.1	✔	✔				
R1PC302B.2				✔		
R1PC302B.3			✔			
R1PC302B.4			✔			✔
R1PC302B.5			✔			✔

PROGRAM EDUCATIONAL OBJECTIVES

The Post Graduates of Computer Application shall:

PEO1: be able to successfully pursue research in Computer Applications and allied disciplines at institutions of transnational reputation.
PEO2: serve in technical or managerial roles at Government firms, Corporates and contributing to the society as successful entrepreneurs through startup.
PEO3: be engaged with leading Global Software services companies, Consultancy organizations and academic Institutions.

PROGRAM OUTCOMES (POs):

1. Computational Knowledge: Apply knowledge of computing fundamentals, computing specialization, mathematics, and domain knowledge appropriate for the computing specialization to the abstraction and conceptualization of computing models from defined problems and requirements.

2. Problem Analysis: Identify, formulate, research literature, and solve complex computing problems reaching substantiated conclusions using fundamental principles of mathematics, computing sciences, and relevant domain disciplines.

3. Design /Development of Solutions: Design and evaluate solutions for complex computing problems, and design and evaluate systems, components, or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.

4. Conduct investigations of complex Computing problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern Tool Usage: Create, select, adapt and apply appropriate techniques, resources, and modern computing tools to complex computing activities, with an understanding of the limitations.

6. Professional Ethics: Understand and commit to professional ethics and cyber regulations, responsibilities, and norms of professional computing practices.

7. Life-long Learning: Recognize the need, and have the ability, to engage in independent learning for continual development as a computing professional.

8. Project management and finance: Demonstrate knowledge and understanding of the computing and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

9. Communication Efficacy: Communicate effectively with the computing community, and with society at large, about complex computing activities by being able to comprehend and write effective reports, design documentation, make effective presentations, and give and understand clear instructions.

10. Societal and Environmental Concern: Understand and assess societal, environmental, health, safety, legal, and cultural issues within local and global contexts, and the consequential responsibilities relevant to professional computing practices.

11. Individual and Team Work: Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary environments.

12. Innovation and Entrepreneurship: Identify a timely opportunity and using innovation to pursue that opportunity to create value and wealth for the betterment of the individual and society at large.

PROGRAMME SPECIFIC OUTCOME (PSO):

The students of Computer Application shall:

PSO1: Have the ability to work with contemporary technologies in computing requisite to Industry 4.0 developing and implementing solutions to real life problems.

PSO2: Demonstrate application development skills learned through technical training and projects to solve real world problems.

COURSE ARTICULATION MATRIX

The Course articulation matrix indicates the correlation between Course Outcomes and Program Outcomes and their expected strength of mapping in three levels (low, medium, and high).

COs#/ POs	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO1 0	PO11	PO1 2	PSO 1	PSO 2
302B.1	1	-	-	-	-	-	2	1	-	-	-	-	2	2
302B.2		2	-	-	-	-	-	-	-	1	-	-	2	2
302B.3	-	3	-	2	-	2	1	-	-	-	1	-	2	2
302B.4	-	-	3	-	-	2	-	-	-	-		-	2	2
302B.5	-	1	2	-	-	2	-	-	1	-		1	2	2

Note: 1-Low, 2-Medium, 3-High

COURSE ASSESSMENT

S. No.	Assessment Tools	CIE					TOTAL CIE Marks	SEE
		AI- 1	MTE	AT-2	LAB	LAB Exam		
1	Theory	25	50	25			100	100
	Lab				25	25	50	100

COURSE CONTENT : THEORY+ PRACTICAL

THEORY	**Introduction to Algorithms** **8 hours** Introduction: Algorithms, Analysis of algorithms, Asymptotic Notations, Growth of function, Recurrences, and their solution methods. Sorting in polynomial Time: Insertion sort, Merge sort, Heap sort, and Quick sort Sorting in Linear Time: Counting sort, Radix Sort, Bucket Sort. **Advance Data Structure** **8 hour** Advanced Data Structure: Red Black Trees, Binomial Heap, B-Tree, Fibonacci Heap, and priority Queues. **Advance Design and Analysis Techniques** **8 hours** Dynamic programming: Matrix Multiplication and Longest Common Sequence. Greedy Algorithm: Huffman Coding, and Knapsack problem Backtracking: N-Queen Problem, Sum of subset problem **Graph Algorithms** **8 hours** Elementary Graph Algorithms, Breadth First Search, Depth First Search, Minimum Spanning Tree: Kruskal's Algorithms, Prim's Algorithms, Single Source Shortest Path, All pair Shortest Path, and Traveling Salesman Problem **Advanced Algorithms Design** **8 hours** Randomized Algorithms, String Matching, NP-Hard and NP-Completeness Approximation Algorithms, Matrix Operations.

Lesson Plan for Integrated Courses

FOR THEORY 15 weeks * 3 Hours = 45 Classes) (1credit = 1Lecture Hour)
FOR PRACTICAL 15 weeks * 2Hours = 30 Hours lab sessions (1 credit = 2 lab hours)

Lesson Plan for Theory

S.No.	Topic	Lesson Outcome (LO)	Level (SOLO)	CO Mapping	Assessment	Learning Activity
1	Introduction to Algorithms	Define an algorithm and explain the properties and importance of algorithms in problem-solving.	Relational	CO1, CO2	MCQ test	Think-Pair-Share
2	Analysis of Algorithms	Analyze algorithms based on input size and understand time and space complexities.	Relational	CO1,CO2	Wooclap and Interactive Quiz	Brain stroming and Think Pair Share
3	Asymptotic Notations	Illustrate and compare the Big-O, Omega, and Theta notations with examples.	Relational	CO1, CO2	Brain stroming and Presentation	Group Discussion Activity
4	Growth of Functions	Demonstrate understanding of function growth and use it to analyze algorithm performance.	Relational	CO1	MCQ-based quiz	Wooclap quiz on segment definitions and applications

5	Recurrence Relations	Formulate recurrence relations for recursive algorithms.	Multistructural	CO1, CO2	Case study analysis	Wooclap interactive quiz
6	Substitution Method	Solve recurrence relations using the substitution method with mathematical induction.	Relational	CO1, CO2, CO3	Think-Pair-Share	Short reflection paper
7	Recursion Tree Method	Solve recurrences using the recursion tree technique to estimate algorithm complexity.	Relational	CO2	Group presentation & peer evaluation	Mentimeter (Voting)
8	Master's Theorem	Apply Master's Theorem to solve divide-and-conquer recurrences.	Relational	CO2	Scenario-based MCQ test	Think-Pair-Share (Compare Row Major Order, and Column Major Order)
9	Insertion Sort	Implement insertion sort and analyze its best, worst, and average-case time complexities.	Relational	CO1, CO2	Case study review	Think-Pair-Share
10	Merge Sort	Design and implement merge sort using the divide-and-conquer paradigm.	Relational	CO1, CO2	Group presentation & peer evaluation	Group risk assessment simulation
11	Heap Sort	Use heap data structures to perform sorting and analyze its efficiency.	Relational	CO2	Report submission	Gallery Walk (Exploring different case studies)
12	Quick Sort	Construct quick sort algorithm with randomized and deterministic pivots and analyze performance.	Relational	CO2	Peer assessment	Case study on Linked lists
13	Counting Sort	Implement counting sort and explain its linear time complexity for specific input types.	Relational	CO1, CO2	Wooclap Quiz	Brain stroming ,Think Pair Share
14	Radix Sort	Demonstrate the working of radix sort and discuss its stability and efficiency.	Relational	CO1, CO2	Brain stroming	Group Discussion Activity
15	Bucket Sort	Apply bucket sort for uniformly distributed input and analyze its complexity.	Relational	CO1, CO2	Wooflash quiz	Jigsaw activity (Teams research different

						players and present their role)
16	Red-Black Trees	Explain properties of red-black trees and perform insertions and deletions maintaining balance.	Relational	CO2	Report submission	Gallery Walk (Exploring different case studies)
17	Binomial Heap	Describe the structure of a binomial heap and perform basic heap operations.	Relational	CO2	Peer assessment	Case study on Linked lists
18	B-Trees	Understand B-Trees and implement insertion, deletion, and search operations.	Relational	CO1, CO2	Wooclap Quiz	Brain stroming ,Think Pair Share
19	Fibonacci Heap	Illustrate Fibonacci heap structure and demonstrate amortized cost of its operations.	Relational	CO1, CO2	Brain stroming	Group Discussion Activity
20	Priority Queues	Implement priority queues using heaps and analyze performance in different scenarios.	Relational	CO1, CO2	Wooflash quiz	Jigsaw activity (Teams research different players and present their role)
21	Matrix Chain Multiplication (DP)	Solve matrix chain multiplication problems using dynamic programming to minimize computations.	Relational	CO1, CO2	MCQ test	Think-Pair-Share
22	Longest Common Subsequence (DP)	Apply dynamic programming to determine the longest common subsequence between two sequences.	Relational	CO1, CO2	Group debate	Wooclap quiz
23	Greedy Algorithms Introduction	Understand the greedy strategy and identify problems suitable for greedy solutions.	Relational	CO2	Group debate	Think-Pair-Share
24	Huffman Coding	Construct optimal prefix codes using Huffman's algorithm and explain its compression benefits.	Relational	CO1, CO2	Reflective paper	Wooflash quiz
25	0/1 Knapsack Problem (Greedy)	Apply greedy approach to solve fractional knapsack and explain its limitations on 0/1 knapsack.	Extended abstract	CO1, CO2	Reflective paper	Group discussion

26	Backtracking Introduction	Explain the concept of backtracking and identify problem domains suitable for it.	Relational	CO1, CO2	Group debate	Think-Pair-Share
27	N-Queens Problem	Formulate and solve the N-Queens problem using backtracking strategies.	Relational	CO1, CO2	Reflective paper	Case study gallery walk
28	Subset Sum Problem	Implement backtracking to determine if a subset exists that adds up to a given sum.	Relational	CO1, CO2	Group presentation	Jigsaw activity
29	Graph Representation Techniques	Represent graphs using adjacency matrix and adjacency list with applications.	Relational	CO1, CO2	Case study review	Think-Pair-Share
30	Breadth First Search (BFS)	Apply BFS to traverse graphs and analyze its complexity and applications.	Relational	CO2	Group case study	Wooclap quiz
31	Depth First Search (DFS)	Implement DFS traversal and explain its use in cycle detection and component discovery.	Multistructural	CO2	Data analysis project	Wooflash Q&A
32	Minimum Spanning Trees Overview, Kruskal's Algorithm	Construct minimum spanning trees using Kruskal's algorithm and analyze its complexity.	Multistructural	CO1, CO2	Position paper	Debate
33	Prim's Algorithm	Implement Prim's algorithm to find minimum spanning trees and compare with Kruskal's.	Relational	CO1, CO2	Group debate	Wooclap quiz
34	Single Source Shortest Path Overview, Dijkstra's Algorithm	Apply Dijkstra's algorithm to determine shortest paths from a single source in weighted graphs.	Relational	CO1, CO2, CO4	Group case study	Wooclap quiz
35	Bellman-Ford Algorithm	Use Bellman-Ford algorithm to find shortest paths and detect negative weight cycles.	Relational	CO1, CO2, CO4	Group discussion	Think-Pair-Share
36	All-Pairs Shortest Path	Formulate solutions for all-pairs shortest paths using matrix-based or graph algorithms.	Relational	CO1, CO2, CO4	Timeline project	Wooflash quiz
37	Floyd-Warshall Algorithm	Implement Floyd-Warshall algorithm and analyze its time and space complexities.	Relational	CO1, CO2, CO4	Group discussion	Think-Pair-Share

38	Traveling Salesman Problem	Formulate the Traveling Salesman Problem and explore brute-force and heuristic solutions.	Relational	CO1, CO2, CO4	Group Discussion & Comparison	Wooclap Q&A
39	Randomized Algorithms Overview	Understand the concept and advantages of randomized algorithms through basic examples.	Unistructural	CO1,CO4	Case study report	Jigsaw activity
40	String Matching Algorithms	Compare various string matching approaches and analyze their performance.	Relational	CO1,CO4	Case study review	Think-Pair-Share
41	Naive String Matching	Implement the naive pattern matching algorithm and evaluate its limitations.	Extended Abstract	CO1,CO4	MCQ	Wooclap quiz
42	Knuth-Morris-Pratt (KMP) Algorithm	Apply the KMP algorithm for efficient string matching using prefix function.	Extended Abstract	CO1,CO4	MCQ	Group Discussion & Comparison with DFS
43	NP-Hard and NP-Completeness	Explain the concept of computational complexity, P vs NP, and classify problems accordingly.	Relational	CO1, CO2	Group case study	Wooclap quiz
44	Approximation Algorithms	Describe the need for approximation algorithms and apply them to NP-Hard problems.	Relational	CO1,CO2, CO3	Research paper on binary search	Think-Pair-Share
45	Matrix Operations	Perform and analyze basic matrix operations relevant to algorithm design.	Relational	CO1, CO3, CO4	MCQ	Group discussion

Lesson Plan for Practical's

PRACTICAL	Write a program to sort given set of numbers in ascending/descending order using Bubble sort and also search a number using binary search.
1	Write a program to sort a given set of numbers in ascending/descending order using Insertion sort and also search a number using linear search.
2	Write a program to sort a given set of numbers in ascending/descending order using Quick sort and any other sorting algorithm. Also record the time taken by these two programs and compare them.
3	Write a program to sort a given set of numbers using Heap sort.
4	Write a program to sort a given set of numbers using Heap sort.
5	Write a program to sort a given set of numbers Counting Sort.
6	Write a program to sort a given set of numbers Radix Sort.
7	Write a program to implement Matrix Chain Multiplication.
8	Implement Matrix Chain Multiplication (MCM) using Dynamic Programming.
9	Implement Longest Common Subsequence (LCS) using Dynamic Programming.
10	Implement BFS for a given Graph G.
11	Implement DFS for a given Graph G.
12	Implement the Minimum Spanning tree using Prims's Approach.
13	Implement the Minimum Spanning tree using Kruskal Approach.
14	Write a program to implement Greedy algorithm using Acitivity Selection Problem.

T= Theory, L= Lab

BIBLIOGRAPHY

- **Text Book**
  1. Thomas H. Coreman, Charles E. Leiserson and Ronald L. Rivest, "Introduction to Algorithms", The MIT Press, 3rd edition, 2009.

- **Reference Books**
  1. Ellis Horowitz, SartajSahni, SanguthevarRajasekaran. Fundamentals of Computer Algorithms, MIT Press, Second Edition (Indian reprint: Prentice-Hall), 2008.
  2. Aho, Hopcraft, Ullman, "The Design and Analysis of Computer Algorithms" Pearson Education.

- **Journals/Magazines/Govt. Reports/Gazatte/Industry Trends**
  1. **https://leetcode.com**
- **Webliography**
  1. https://www.vssut.ac.in/lecture_notes/lecture1428551142.pdf
  2. https://iare.ac.in/sites/default/files/lecture_notes/IARE_SE_Lecture%20Notes.pdf

- **SWAYAM/NPTEL/MOOCs Certification**
  1. Code Tantra
  2. Coding Ninjas
  3. Code Chef
  4. https://onlinecourses.nptel.ac.in/noc19_cs47/preview (Swayam)

PROBLEM-BASED LEARNING

Exercises in Problem-based Learning (Assignments) (Min 45 Problems*)

S.No.	Problem	BTL
1	Write a program to implement Minimum Cost spanning tree.	BTL5
2	Write a program to implement Sum of subset problem.	BTL5
3	Write a program to implement Greedy algorithm using Task Scheduling Problem.	BTL5
4	Write a program to implement Greedy algorithm using Activity Selection Problem.	BTL5
5	Compute the transitive closure of a given directed graph using Warshall's algorithm. Write a program to implement shortest path algorithm.	BTL6
6	Write a program to find all Hamiltonian Cycles in a connected undirected Graph G of n vertices using backtracking principle.	BTL5
7	Develop a program to implement solve LCS problem.	BTL6
8	Develop a program to implement Huffman-code.	BTL6
9	Find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm. Use Union-Find algorithms in your program.	BTL6
10	Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.	BTL6
11	Design a programs to (a) Implement All-Pairs Shortest Paths problem using Floyd's algorithm. (b) Implement Travelling Sales Person problem using Dynamic programming.	BTL6

12	Design and implement to find a subset of a given set S = {Sl, S2, ,Sn} of n positive integers whose SUM is equal to a given positive integer d. For example, if S ={1, 2, 5, 6, 8} and d= 9, there are two solutions {1,2,6}and {1,8}. Display a suitable message, if the given problem instance doesn't have a solution.	BTL6
13	Comparing Kruskal and Prim's algorithm in MST.	BTL5
14	Greedy and Backtracking Algorithm Comparison in Graph Coloring Problem.	BTL5
15	Build the game of Capsa Susun using Greedy Algorithm	BTL6
16	Solving Travelling Salesman Problem Using Greedy Algorithm and Brute Force Algorithm	BTL3
17	Design and Analysis 0/1 Knapsack Problem.	BTL6
18	Design Coin Change Problem using Brute Force and Greedy Algorithm	BTL6
19	Comparing Exhaustive Search Algorithm and Greedy Algorithm in job Scheduling Problem	BTL5
20	Build a Cash Flow Minimiser	BTL6
21	Build a CB Mario game using Dynamic Programming Optimisation.	BTL6
22	Build an application of Sudoku game using Backtracking method	BTL6
23	Build a Snakes & Ladders game, challenge the player to win in minimum number of moves. You can use BFS to compute it.	BTL6
24	Explain (0/1) knapsack problem and fractional knapsack problem.	BTL5
25	Explain the Dynamic Programming (DP) Algorithmic Paradigm? List a few problems which can be solved using the same.	BTL5
26	Explain the recursive algorithms? State the important rules which every recursive algorithm must follow.	BTL3
27	How can we compare between two algorithms written for the same problem?	BTL3
28	Design an algorithm that finds the number of ways in which you can traverse N meters by doing jumps of 1, 2, 3, 4, or 5- meter lengths. Assume that N can be a very large number. What is the resulting complexity?	BTL6
29	Design an algorithm that finds all pairs of integer arrays whose sum is equal to a given number?	BTL3
30	Suppose a given space S of N contiguous memory cells is allocated to 6 stacks. Describe ways that the stack may be maintained in S	BTL4
31	Write an algorithm to find solution to the Towers of Hanoi problem. Explain the working of your algorithm with 4 disks using diagrams.	BTL4
32	Give an algorithm for graph coloring problem and analyze its complexity	BTL5
33	Construct a B-tree of minimum degree 3 by inserting the elementsin the order given F, Q,P,K,A,L,R,M,N,X,Y,D,Z,E,H,T,V,W,C. from the constructed tree delete A,P,Q,R,T.	BTL6
34	Construct a Red Black tree by inserting 9,19,30,15,16 and 27 into an initially empty tree and also delete 15,16 and 30 from the tree	BTL6

35	Obtain the solution to knapsack problem by Dynamic Programming method n=6, (p1, p2,...p6)=(w1,w2,...w6)=(100,50,20,10,7,3) and m=165.	BTL5
36	Apply Merge Sort to sort the list a[1:10]=(31,28,17,65, 35, 42.,86,25,45,52). Draw the tree of recursive calls of merge sort, merge functions	BTL6
37	Solve the Travelling Salesman problem Dynamic programming with example.	BTL6
38	Write an algorithm String Matching using Rabin-Karp algorithm and also analyze its complexity.	BTL6
39	Write an algorithm for randomized quick sort algorithm.	BTL6
40	Write the Prim's Algorithm to find out Minimum Spanning Tree. Apply the same and find MST for the graph given below.	BTL6